

Reflexión

Hacer este ACA realmente sí me pareció muy fascinante ya que no lo tomé como solo un entregable sino por el contrario como un producto que debe tener inicio y debe tener fin, claramente no se me está solicitando hacer un juego, pero!, si buscó buenas prácticas de creación, estructura y organización para la creación de un video juego, claro, falta mucho como bases de datos GDD entre muchas cosas, pero quería estructurar un proyecto que tenga una vista seria y responsable.

Ya en materia de aprendizaje comprendí que el diseño de un videojuego comienza mucho antes de escribir código. Al iniciar el trabajo tenía varias dudas sobre los entregables solicitados, especialmente porque confundía el diagrama UML con el pseudocódigo. A medida que desarrollé la actividad entendí que el UML permite representar la estructura del videojuego mediante clases, atributos, métodos y relaciones, mientras que el pseudocódigo describe la lógica del funcionamiento del juego y del Game Loop sin depender de un lenguaje de programación.

Otro aspecto nuevo para mí fue la tabla de responsabilidades por clase. Antes de esta actividad no conocía este tipo de documento ni su importancia dentro del diseño orientado a objetos. Al elaborarla comprendí que ayuda a definir claramente la función de cada clase, facilitando la organización del proyecto y la creación del UML y evitando mezclar responsabilidades entre las diferentes entidades del videojuego.

¿Qué vamos a cambiar de nuestro diseño?

Desde una perspectiva autocrítica y con el fin de seguir mejores prácticas en ingeniería de software, si tuviéramos que rediseñar el proyecto, realizaríamos los siguientes cambios:

1. Estructura de Poderes (PowerUps): En el diseño actual, los poderes se identifican a través de un atributo 'type' (de tipo string). Para seguir el principio Abierto/Cerrado (Open/Closed Principle) de SOLID, implementaremos una jerarquía de clases donde cada poder sea una subclase de una clase base 'PowerUp'. Esto evitará estructuras condicionales complejas a medida que se añadan más tipos de poderes.
2. Desacoplamiento del Sistema de Física: Actualmente la lógica de rebotes y colisiones está distribuida directamente en 'Ball', 'Paddle' y 'Brick'. Consideramos que será mucho más limpio y modular implementar un 'CollisionManager' o un sistema de física dedicado, reduciendo la dependencia directa entre estas entidades y facilitando futuras modificaciones en el comportamiento de colisión.

3. Modularidad en la UI: Separaríamos de forma más estricta la representación de datos del juego del renderizado visual de la interfaz. Esto facilitaría implementar múltiples pieles (skins) o cambiar la resolución del juego sin afectar la lógica interna.